

Lesson 08: Terraform Locals

Learning Objectives

- Understand local values and when to use them over variables
- Define locals blocks with expressions
- Use locals for computed values, name prefixes, and tag normalization
- Reduce repetition with locals

1. What are Local Values?

Local values (also called **locals**) are named expressions that you define once and reference many times within a configuration. Unlike variables, locals are **not settable by the user** — they are computed entirely within the configuration.

```
locals {
  name_prefix = "myapp-${var.environment}"
  common_tags = {
    Environment = var.environment
    ManagedBy   = "Terraform"
    Project     = var.project_name
  }
}
```

Variables vs Locals

	Variables	Locals
Set by user?	Yes (tfvars, CLI)	No (computed internally)
Reference syntax	<code>var.name</code>	<code>local.name</code>
Use case	User-supplied parameters	Computed intermediate values
Can contain logic?	Limited (validation only)	Full expressions (interpolation, conditionals, functions)

The pattern is:



2. Defining Locals

All locals are defined in a single `locals` block (you can have multiple blocks — they merge):

```
locals {
  # String interpolation
  name_prefix = "${var.project_name}-${var.environment}"

  # Conditional
  bucket_name = var.custom_bucket_name != "" ? var.custom_bucket_name :
"${local.name_prefix}-default"

  # Computed list
  supported_regions = ["us-east-1", "eu-west-1"]

  # Computed map
  common_tags = {
    Name          = local.name_prefix
    Environment    = var.environment
    ManagedBy     = "Terraform"
  }

  # Function results
  timestamp = timestamp()

  # Arithmetic
  retention_days = var.retention_days * 86400

  # Merge maps
  all_tags = merge(local.common_tags, var.extra_tags)
}
```

3. Using Locals in Resources

Reference locals with `local.<name>`:

```
locals {
  name_prefix = "${var.project_name}-${var.environment}"
  common_tags = {
    Environment = var.environment
    ManagedBy   = "Terraform"
  }
}

resource "aws_s3_bucket" "main" {
  bucket = "${local.name_prefix}-data"
  tags   = local.common_tags
}

resource "aws_dynamodb_table" "main" {
  name = "${local.name_prefix}-table"
```

```
billing_mode = "PAY_PER_REQUEST"
hash_key     = "pk"

attribute {
  name = "pk"
  type = "S"
}

tags = local.common_tags
}

resource "aws_sqs_queue" "main" {
  name = "${local.name_prefix}-events"
  tags = local.common_tags
}
```

All three resources share the same name prefix and tags, defined once in `locals`.

4. Common Local Patterns

Name Prefix Pattern

```
locals {
  name_prefix = "${var.project_name}-${var.environment}"
}

resource "aws_s3_bucket" "main" {
  bucket = "${local.name_prefix}-bucket"
}

resource "aws_dynamodb_table" "main" {
  name = "${local.name_prefix}-table"
}

resource "aws_sqs_queue" "main" {
  name = "${local.name_prefix}-queue"
}
```

Tag Normalization

```
locals {
  required_tags = {
    Environment = var.environment
    ManagedBy   = "Terraform"
    Project     = var.project_name
  }

  # Merge required tags with optional user-provided tags
```

```
all_tags = merge(local.required_tags, var.extra_tags)
}
```

Conditional Locals

```
locals {
  # If environment is prod, use larger config
  is_prod      = var.environment == "prod"
  bucket_name  = local.is_prod ? "prod-data-bucket" :
"${var.environment}-data-bucket"

  # Conditional encryption
  enable_encryption = var.environment != "dev"

  # Region-specific config
  sse_algorithm = var.region == "us-east-1" ? "AES256" : "aws:kms"
}
```

Computed Values from Functions

```
locals {
  # Upper/lower case normalization
  normalized_name = lower(var.project_name)

  # Substrings
  short_region = substr(var.region, 0, 2)

  # CIDR calculations
  vpc_cidr = "10.0.0.0/16"
  subnet_cidrs = cidrsubnets(local.vpc_cidr, 4, 4, 4)
}
```

5. Locals with **merge** and **lookup**

```
locals {
  # Base configuration
  base_config = {
    engine = "standard"
    timeout = 30
  }

  # Environment overrides
  env_overrides = {
    dev = { timeout = 10 }
    prod = { timeout = 120 }
  }
}
```

```
}

# Merged config
final_config = merge(
  local.base_config,
  lookup(local.env_overrides, var.environment, {})
)
}
```

6. Complete Example

```
# variables.tf
variable "environment" {
  description = "Deployment environment"
  type        = string
  default     = "dev"
}

variable "project_name" {
  description = "Project name"
  type        = string
  default     = "myapp"
}

variable "extra_tags" {
  description = "Additional tags"
  type        = map(string)
  default     = {}
}

# locals.tf
locals {
  name_prefix = "${var.project_name}-${var.environment}"
  is_prod     = var.environment == "prod"

  common_tags = {
    Environment = var.environment
    ManagedBy   = "Terraform"
  }

  all_tags = merge(local.common_tags, var.extra_tags)

  # Conditional configuration
  bucket_name     = "${local.name_prefix}-storage"
  table_name      = "${local.name_prefix}-db"
  queue_name      = "${local.name_prefix}-events"
  retention_days  = local.is_prod ? 365 : 30
}

# main.tf
```

```
resource "aws_s3_bucket" "main" {
  bucket = local.bucket_name
  tags    = local.all_tags
}

resource "aws_dynamodb_table" "main" {
  name           = local.table_name
  billing_mode   = "PAY_PER_REQUEST"
  hash_key       = "pk"

  attribute {
    name = "pk"
    type = "S"
  }

  tags = local.all_tags
}

resource "aws_sqs_queue" "main" {
  name                           = local.queue_name
  message_retention_seconds     = local.retention_days * 86400
  tags                          = local.all_tags
}

# outputs.tf
output "bucket_name" {
  value = aws_s3_bucket.main.id
}

output "table_name" {
  value = aws_dynamodb_table.main.name
}

output "queue_name" {
  value = aws_sqs_queue.main.name
}

output "tags_applied" {
  value = local.all_tags
}

# terraform.tfvars

environment = "prod"
project_name = "myapp"
extra_tags = {
  Owner = "Alice"
  Team  = "DevOps"
}
```

7. Key Takeaways

- Locals are **computed once, referenced many times** — they reduce repetition
- Use `local.<name>` to reference a local value (not `var.<name>`)
- Locals can contain any Terraform expression: interpolation, conditionals, functions, arithmetic
- Common patterns: name prefixes, tag normalization, conditional configuration
- Multiple `locals` blocks are merged automatically by Terraform
- Locals are **not** user-configurable — they are derived from variables and other inputs
- The typical flow: `variables` → `locals` → `resources` → `outputs`

example and exercise repo url

<https://github.com/eyu-se/terraform-training-material-with-projects>

Trainer - Eyuel M.